

Onusuite : audit du code, de la sécurité et de l'intégrité métier

Lecture critique du dépôt `compta` (branche `commercial-hr-paie`) : ERP SaaS multi-sociétés couvrant comptabilité, commercial, trésorerie, stock, production, RH/paie, projets et documents. Audit statique complet, exécution du typecheck, du lint, des tests unitaires et de `npm audit`.

DATE	CODE SOURCE	BASE DE DONNÉES	REQUÊTES CLIENT
3 septembre 2026	395 fichiers TS/TSX · ≈102 000 lignes	83 scripts SQL · ≈289 tables	1 125 fonctions · 1 041 <code>select</code>

TESTS
2 212 unitaires · 127
en échec

Verdict. Le produit est fonctionnellement très large et le socle technique moderne (React 19, Vite 8, TypeScript strict qui passe, i18n trilingue à parité). Mais la couche de sécurité multi-tenant est incomplète et en partie illusoire : **129 tables créées après le correctif RLS n° 37 restent en accès libre** (`USING (true)`), dont des données de santé et de paie ; le tenant actif repose sur une variable de session Postgres non fiable derrière le pooler ; et **un mot de passe Postgres de production est en clair dans un fichier versionné**. Côté métier, les invariants comptables (équilibre, immutabilité des écritures validées, unicité des numéros) et les calculs de paie ne sont pas garantis par la base. Le CI est rouge et le déploiement ne l'attend pas.

La bonne nouvelle : la plupart des correctifs sont mécaniques et localisés. Les cinq points critiques peuvent être fermés en une semaine de travail concentré.

Synthèse

SÉVÉRITÉ	NOMBRE	CE QUE ÇA SIGNIFIE
CRITIQUE	5	Fuite de données inter-sociétés ou compromission possible aujourd'hui, sans exploit sophistiqué.
ÉLEVÉ	8	Résultats comptables ou de paie faux, fonctionnalités silencieusement cassées, pipeline de livraison non fiable.
MOYEN	11	Dette d'architecture, performance, hygiène du dépôt, tests qui ne prouvent rien.
FAIBLE	5	Corrections rapides, cohérence, accessibilité.

CONTRÔLE EXÉCUTÉ	RÉSULTAT	DÉTAIL
<code>tsc --noEmit</code>	PASSE	Aucune erreur de type.
<code>oxlint</code>	ÉCHOUÉ	14 avertissements ; le CI exige <code>--max-warnings=0</code> .
<code>vitest run</code>	ÉCHOUÉ	127 tests en échec sur 2 212 (4 fichiers).

CONTRÔLE EXÉCUTÉ	RÉSULTAT	DÉTAIL
npm audit	7 HIGH	xlsx (sans correctif), pdfjs-dist, react-router, postcss, nanoid, undici.
scripts/check-i18n.mjs	PASSE	Parité fr/en/ar sur tous les namespaces.
Bundle de production	LOURD	Chunk principal 3,39 Mo, worker PDF 1,26 Mo, aucun chargement différé.

Critique : à traiter cette semaine

SEC-01	CRITIQUE	Mot de passe Postgres de production en clair dans un fichier versionné
CONSTAT	Le script de migration se connecte au pooler Supabase avec un mot de passe littéral. Le fichier est suivi par git et le mot de passe apparaît dans l'historique (commit 31b79bd). Le même dépôt suit aussi supabase/.temp/ (référence du projet) et mirror-daemon/config.json malgré le .gitignore .	
PREUVE	app/run-sql-migrations.mjs:9-15	
IMPACT	Quiconque a ou a eu accès au dépôt (collaborateur, fork, fuite de laptop, IA connectée au repo) possède un accès postgres complet à la base de production, RLS incluse.	
CORRECTIF	• Changer immédiatement le mot de passe de la base dans Supabase, puis les clés API si le fichier a quitté la machine. • Lire la connexion depuis process.env.DATABASE_URL et faire échouer le script si elle est absente. • Purger l'historique (git filter-repo --path app/run-sql-migrations.mjs --invert-paths puis réécriture) et forcer la mise à jour des clones. • Retirer du suivi supabase/.temp/ , mirror-daemon/config.json , coverage/ et mirror-data/ (git rm --cached). • Ajouter gitLeaks en pre-commit et dans ci.yml ; le grep actuel du job security ne détecte pas ce cas (pattern trop étroit, et continue-on-error).	

SEC-02	CRITIQUE	129 tables en accès libre : le correctif RLS n° 37 n'a pas été réappliqué aux sprints suivants
CONSTAT	La migration 37_security_fix_all_allow_all_policies.sql remplace dynamiquement les pc 64 créent ensuite des dizaines de tables avec CREATE POLICY allow_all_... FOR ALL USING(tru noms de politiques donne 129 tables sans aucune politique tenant . Parmi elles : work_stoppages , medical_exams , ijss_history , social_declarations , dsn_payroll_variable_elements , employee_documents , employee_exit_processes , leave_bal document_shares , electronic_signatures , sepa_payment_orders .	
PREUVE	app/sql/52_sprint_f_crm_sales.sql:34 · app/sql/62_sprint_f_social_declarations.sql:35 · complète obtenue par grep -E "USING\s*(\\s*true" sur sql/3[9]-6*.sql .	
IMPACT	Tout utilisateur authentifié d'une société quelconque peut lire, modifier et supprimer les arrêts m sociales de toutes les sociétés en appelant PostgREST directement avec la clé publiable (celle qu n'est pas un contrôle : l'attaquant ne l'envoie pas. Si le rôle anon conserve les grants par défaut, RGPD caractérisée (données de santé, article 9).	

CORRECTIF

Une migration `74_rls_generic_tenant_isolation.sql` qui généralise la boucle du n° 37 à *toutes* les tables en CI :

```
DO $$
DECLARE r record; pol record;
BEGIN
  FOR r IN
    SELECT c.table_name
    FROM information_schema.columns c
    WHERE c.table_schema = 'public' AND c.column_name = 'tenant_id'
  LOOP
    EXECUTE format('ALTER TABLE %I ENABLE ROW LEVEL SECURITY', r.table_name);
    EXECUTE format('ALTER TABLE %I FORCE ROW LEVEL SECURITY', r.table_name);
    FOR pol IN SELECT polycname FROM pg_policies
      WHERE tablename = r.table_name AND polycname LIKE 'allow_all%' LOOP
      EXECUTE format('DROP POLICY %I ON %I', pol.polycname, r.table_name);
    END LOOP;
    EXECUTE format('CREATE POLICY %I ON %I FOR SELECT USING (tenant_id = current_tenant_id)', r.table_name, r.table_name);
    EXECUTE format('CREATE POLICY %I ON %I FOR INSERT WITH CHECK (tenant_id = current_tenant_id)', r.table_name, r.table_name);
    EXECUTE format('CREATE POLICY %I ON %I FOR UPDATE USING (tenant_id = current_tenant_id)', r.table_name, r.table_name);
    EXECUTE format('CREATE POLICY %I ON %I FOR DELETE USING (tenant_id = current_tenant_id)', r.table_name, r.table_name);
  END LOOP;
END $$;
-- Vérification à exécuter en CI : doit renvoyer 0 ligne
SELECT tablename, polycname FROM pg_policies
WHERE schemaname = 'public' AND (qual = 'true' OR with_check = 'true');
```

- Révoquer les grants du rôle `anon` sur toutes les tables métier (`REVOKE ALL ON ALL TABLES FROM anon`).
- Interdire par un test CI toute occurrence de `USING (true)` dans `sql/` hors tables de référence.
- Les tables enfants (`*_lines`, `*_recipients`) sans `tenant_id` doivent recevoir une politique de sécurité.

SEC-03

CRITIQUE

Le tenant actif repose sur une variable de session Postgres, non fiable derrière le pooler et non attendue côté client

CONSTAT

- `set_active_tenant()` fait `set_config('app.active_tenant_id', ..., false)` : portée *session*. PostgREST et le pooler Supabase servent chaque requête HTTP sur une connexion potentiellement différente ; la valeur posée par un appel n'est pas garantie visible par le suivant, et elle peut « traîner » sur la connexion pour la requête d'un autre utilisateur.
- `current_tenant_id()` retombe alors sur « le premier tenant du user par `created_at` », ce qui est arbitraire pour un utilisateur multi-sociétés.
- Côté client, `setTenantId(newTenantId)` n'est pas `await é` : les premières requêtes partent avant que le RPC ait répondu, et l'exception « SECURITY: Tenant verification failed » devient une promesse rejetée non gérée.

PREUVE

`app/sql/27_multi_tenant_switching.sql:26-53, 93-111 · app/src/lib/auth.tsx` (appels `setTenantId(...)` sans `await` dans `loadUser`).

IMPACT

Pour un cabinet comptable ou un groupe gérant plusieurs sociétés : lectures mélangées entre sociétés sur les requêtes sans filtre client, insertions rejetées ou acceptées au hasard par le `WITH CHECK`, comportement impossible à reproduire. Le contrôle d'appartenance dans la fonction empêche la fuite vers un tenant étranger, mais pas le mélange entre les tenants légitimes de l'utilisateur.

CORRECTIF

- Porter le tenant actif dans le JWT : un *Custom Access Token Hook* Supabase qui copie `app_metadata.active_tenant_id` dans les claims, et `current_tenant_id()` qui lit `auth.jwt()` ->> `'active_tenant_id'`. Le changement de société devient un appel RPC qui met à jour `app_metadata` puis `refreshSession()`.
- Alternative plus rapide : envoyer un en-tête `x-tenant-id` (option `global.headers` du client) et le lire dans la fonction via `current_setting('request.headers', true)::json`, toujours validé contre `tenant_users`.
- Supprimer le fallback « premier tenant » : sans tenant explicite, la fonction renvoie `NULL` et RLS ne renvoie rien.
- `await setTenantId()` avant `setUser()`, et afficher une erreur bloquante si la vérification échoue.

SEC-04

CRITIQUE

Des secrets sont préfixés `VITE_` : ils sont compilés dans le bundle dès qu'ils sont référencés

CONSTAT

Le `.env` local contient `VITE_SUPABASE_SECRET_KEY` et `VITE_SUPABASE_JWKS_URL`. `DEPLOY.md` et `.env.example` demandent de configurer `VITE_RESEND_API_KEY` dans Cloudflare Pages. Aucune de ces variables n'est référencée dans `src/` aujourd'hui, mais Vite expose toute variable `VITE_*` à `import.meta.env` : un seul import suffit pour livrer la clé de service Supabase (qui contourne RLS) ou la clé Resend à tous les navigateurs.

PREUVE

`app/.env` (non versionné) · `app/.env.example:18-20` · `DEPLOY.md` section 1 « Variables d'environnement ».

CORRECTIF

- Renommer sans préfixe (`SUPABASE_SECRET_KEY`, `RESEND_API_KEY`) et retirer `VITE_RESEND_API_KEY` de la doc et de l'exemple. Les emails partent déjà d'une edge function, c'est le bon endroit.
- Ajouter au build un contrôle qui échoue si `dist/` contient `sb_secret_`, `re_` ou `service_role`.

DB-01

CRITIQUE

Le schéma de production n'est pas reproductible : trois sources de vérité qui se contredisent

CONSTAT

- `supabase/migrations/20260729194914_remote_schema.sql` fait **0 octet**.
- `supabase-schema.sql` (90 Ko) déclare 85 politiques `allow_all` et ne connaît pas le multi-tenant.
- `run-sql-migrations.mjs` n'exécute que les fichiers `NN_*.sql` : il ignore `multi_tenant_migration.sql`, `legislation_packs_migration.sql`, `audit_schema.sql`, c'est-à-dire les fondations. Les numéros 38, 44 et 71 existent en double.
- Selon le SQL du dépôt, `tenants` n'a *aucune* politique INSERT et `tenant_users` exige d'être déjà admin pour s'insérer. Or `createTenantForUser` insère directement dans ces deux tables depuis le navigateur. Soit l'onboarding est cassé, soit la base de production a des politiques que le dépôt ne décrit pas.

PREUVE

`app/run-sql-migrations.mjs:56-58` · `app/sql/multi_tenant_migration.sql:57, 99-102` · `app/src/lib/queries/misc.ts:857-905`

IMPACT

Impossible de recréer un environnement de staging fidèle, d'auditer réellement les politiques en place, ni de restaurer proprement après incident. L'audit SEC-02 lui-même ne peut être confirmé qu'en interrogeant `pg_policies` en production.

CORRECTIF

- Exporter l'état réel (`supabase db dump --schema public et --data-only` pour les tables de référence), en faire la migration de base `0000_baseline.sql`, puis abandonner `sql/` et `supabase-schema.sql` au profit de `supabase/migrations/` uniquement.
- Ajouter un job CI qui applique toutes les migrations sur un Postgres vierge (`supabase start` ou l'image `supabase/postgres`) puis exécute la requête de vérification de SEC-02 et un jeu de tests pgTAP (un user du tenant A ne lit pas le tenant B).
- Créer les tenants via un RPC `SECURITY DEFINER` `create_tenant_for_current_user(payload)` qui insère tenant + admin + employé dans une transaction, plutôt que trois inserts client non atomiques.

Élevé : résultats faux et fonctionnalités silencieusement cassées

ACC-01 **ÉLEVÉ** Aucun invariant comptable n'est garanti par la base

CONSTAT

- `createJournalEntry` insère l'entête puis les lignes en deux requêtes non transactionnelles : un échec sur les lignes laisse une écriture orpheline. Aucun trigger ne vérifie débit = crédit ; seul `JournalSaisiePage` le fait, avec un simple toast « info ».
- `deleteJournalEntry` et `updateJournalEntry` n'ont aucune garde sur le statut : une écriture validée (`posted`) se supprime après un `window.confirm` . C'est incompatible avec l'irréversibilité exigée par l'article L123-22 du Code de commerce, le FEC et la NF 203, et cela vaut pour toute juridiction où le logiciel prétend tenir une comptabilité probante.
- Aucune vérification de période close avant insertion/modification ; `fiscal_periods.status` existe mais n'est lu nulle part dans les écritures.
- `getNextPieceNumber` lit le dernier numéro puis l'insère : deux utilisateurs simultanés obtiennent le même numéro, et aucune contrainte `UNIQUE` (`tenant_id`, `journal_code`, `piece_number`) ne les arrête.
- `transferGescomToAccounting` ne marque pas les pièces transférées : relancer le transfert duplique les écritures.

PREUVE

`app/src/lib/queries/accounting.ts:166-192`, `998-1017`, `1968-2030`
`· app/src/pages/JournalSaisiePage.tsx:601-613 · app/src/pages/JournalEntriesPage.tsx:54-56`

CORRECTIF

- RPC `post_journal_entry(jsonb)` en transaction : contrôle d'équilibre, période ouverte, numérotation via `UPDATE journals SET next_number = next_number + 1 RETURNING` (verrou de ligne), insertion atomique.
- Trigger `BEFORE UPDATE OR DELETE ON journal_entries` qui lève une exception si `OLD.status = 'posted'` ; l'annulation passe par `generateExtourne` qui existe déjà.
- Trigger de contrôle d'équilibre en `DEFERRABLE INITIALLY DEFERRED` sur `journal_lines`.
- Contraintes d'unicité par tenant sur les numéros de pièces, factures, avoirs, bulletins.
- Colonne `transferred_entry_id` sur les pièces commerciales, et filtre `IS NULL` dans `getGescomTransferData`.

ACC-02 **ÉLEVÉ****Rapports financiers calculés côté client sans pagination : tronqués au-delà de 1 000 lignes**

CONSTAT

Sur 1 041 `select`, aucun `.range()` et 29 `.limit()`. PostgREST plafonne une réponse à `max-rows` (1 000 par défaut sur Supabase). Bilan, balance, grand livre, TVA et export FEC lisent `journal_lines` en entier puis agrègent en JavaScript.

PREUVE

`app/src/lib/queries/accounting.ts:473-486` (bilan) · `:1380` (FEC) · `:1393, 1489, 1563`

IMPACT

Dès qu'une société dépasse 1 000 lignes d'écriture (quelques mois d'activité), le bilan affiché est faux et le FEC exporté est incomplet, sans aucune erreur visible. C'est le type de défaut qui fait perdre un client comptable définitivement.

CORRECTIF

Déplacer les agrégations dans des vues ou fonctions SQL (`v_trial_balance`, `fn_balance_sheet(fy_id)`, `fn_fec(fy_id)`) sous RLS, et paginer toutes les listes (`.range()` avec `count: 'exact'`). Fixer `max-rows` plus bas plutôt que plus haut pour rendre le problème visible en test.

PAY-01 **ÉLEVÉ****La génération de bulletins ignore les grilles fiscales et le contrat réel de chaque salarié**

CONSTAT

- `generatePaySlipsForRun` appelle `calculatePayroll` avec `taxRate: 10`, `hoursPerWeek: 35`, `overtimeHours: 0`, `mealVouchers: 0` en dur, et **sans** `gridLines` : le moteur retombe systématiquement sur les taux français 2024-2025 codés en dur, quel que soit le pack de législation (Djibouti, Maroc, etc.) sélectionné par la société.
- Le numéro de bulletin est `BS-{run}-{3 premières lettres du nom}` : deux salariés « Mohamed A. » et « Mohamed B. » produisent le même numéro.
- Le net est calculé dans le navigateur et stocké tel quel : aucune trace des paramètres utilisés, aucune possibilité de recalcul probant.
- Les calculs de tranches (`calculateBracket`) n'appliquent le taux salarial que sur la tranche courante, sans cumul des tranches inférieures : correct pour un barème progressif, mais la CSG à 98,25 % est codée en dur dans un moteur censé être « data-driven ».

PREUVE

`app/src/lib/queries/payroll.ts:130-172` · `app/src/lib/payroll.ts:45-53, 100-104`

CORRECTIF

Charger la grille active du tenant (`payroll_tax_grids`) et les éléments variables (`payroll_variable_elements`, contrats, heures) avant le calcul ; numéroter par séquence ; persister `calc_inputs_jsonb` et `grid_version` sur chaque bulletin ; à terme, déplacer le calcul dans une fonction SQL ou une edge function pour qu'il soit le même partout.

APP-01 **ÉLEVÉ****Trois RPC appelés n'existent dans aucune migration : les stocks ne bougent jamais**

CONSTAT

`createStockMovement` appelle `increment_stock` / `decrement_stock` et **ignore le résultat** ; `generateRecurringEntry` appelle `generate_recurring_entry`. Aucune de ces fonctions n'est définie dans `sql/` ni dans les dumps de schéma.

PREUVE

`app/src/lib/queries/stock.ts:52-56` · `app/src/lib/queries/accounting.ts:2228-2233`

IMPACT	Chaque entrée/sortie est enregistrée dans <code>stock_movements</code> mais <code>products.stock_quantity</code> ne change pas : alertes de réapprovisionnement, MRP et valorisation sont faux. L'utilisateur ne voit aucune erreur.
CORRECTIF	Créer les fonctions (avec filtre tenant et verrou <code>FOR UPDATE</code>), ou mieux, un trigger <code>AFTER INSERT ON stock_movements</code> qui met à jour <code>stock_quantities</code> par entrepôt. Ne jamais ignorer un <code>{ error }</code> de <code>rpc()</code> . Ajouter un test CI qui compare la liste des <code>rpc('...')</code> du code aux fonctions présentes dans le schéma.

SEC-05 **ÉLEVÉ** **Les liens publics de paiement et de partage exigent un compte, et sont lisibles par tous les comptes**

CONSTAT	Les routes <code>/pay/:token</code> et <code>/shared/:token</code> sont déclarées à l'intérieur de <code>ProtectedLayout</code> : un client externe est renvoyé vers <code>/login</code> . En parallèle, <code>getDocumentShare</code> lit <code>document_shares</code> sans filtre tenant, sur une table <code>allow_all</code> , puis la marque « vue » : n'importe quel utilisateur connecté peut parcourir les partages de toutes les sociétés.
PREUVE	<code>app/src/App.tsx:240, 616-617, 621 · app/src/lib/queries/dematerialisation.ts:127-141</code>
CORRECTIF	Sortir ces deux routes du layout protégé ; servir leur contenu par une edge function <code>public-share</code> qui reçoit le token, le compare à un hash stocké (<code>token_hash</code> , jamais le token en clair), vérifie l'expiration et renvoie uniquement les champs nécessaires avec la clé de service. Aucune politique RLS pour <code>anon</code> sur ces tables.

OPS-01 **ÉLEVÉ** **Le CI est rouge et le déploiement ne l'attend pas**

CONSTAT	<code>vitest run</code> échoue : 119 tests du fichier <code>audit-code-quality.test.ts</code> (règle « 0 as any dans les pages » jamais satisfaite), 3 tests <code>i18n</code> qui figent le nombre de clés à 95, 4 tests <code>calculateEcheance</code> dont le mock ne suit plus l'implémentation, 1 test <code>autoScheduleMOs</code>. <code>oxlint --max-warnings=0</code> échoue sur 14 avertissements (dépendances de hooks manquantes, imports inutilisés). <code>deploy.yml</code> se déclenche sur push <code>main</code> indépendamment de <code>ci.yml</code> ; le job « security-audit » est en <code>continue-on-error</code>.
PREUVE	<code>.github/workflows/deploy.yml:3-6 · .github/workflows/ci.yml jobs unit-tests, security-audit · sortie locale de vitest run</code>
CORRECTIF	Faire dépendre le déploiement de <code>ci-success</code> (<code>workflow_run</code> ou fusion des deux workflows avec <code>needs</code>) ; retirer <code>continue-on-error</code> ; transformer le test « qualité » en règle <code>oxlint</code> ou le supprimer ; corriger les 8 tests réels.

OPS-02 **ÉLEVÉ** **La CSP diverge entre quatre fichiers et bloque des appels légitimes**

CONSTAT	Les en-têtes de sécurité sont dupliqués dans <code>netlify.toml</code> (racine et <code>app/</code>), <code>public/_headers</code> et <code>deploy/nginx.conf</code> , avec des <code>connect-src</code> différents. Aucun n'autorise <code>api.frankfurter.app</code> (taux de change), <code>ipwho.is</code> (landing), l'ingest Sentry ni <code>wss://*.supabase.co</code> (realtime). <code>COEP: require-corp</code> est posé sur <code>/*</code> dans Netlify mais seulement sur <code>/dashboard/*</code> dans <code>_headers</code> .
PREUVE	<code>app/src/lib/currencyRates.ts:103 · app/src/pages/LandingPage.tsx:262 · app/public/_headers · deploy/nginx.conf:32</code>

IMPACT	Le rafraîchissement des taux échoue silencieusement en production (le <code>catch</code> avale l'erreur) et les rapports multi-devises utilisent des taux périmés. L'appel à <code>ipwho.is</code> transmet l'adresse IP de chaque visiteur à un tiers sans consentement : à supprimer plutôt qu'à autoriser (utiliser <code>navigator.language</code> ou l'en-tête <code>CF-IPCountry</code> côté Cloudflare).
CORRECTIF	Garder une seule cible de déploiement (Cloudflare d'après <code>DEPLOY.md</code>) et un seul fichier <code>_headers</code> ; supprimer <code>netlify.toml</code> ×2 et <code>nginx.conf</code> ou les générer depuis une source commune. Faire passer les taux par l'edge function <code>refresh-exchange-rates</code> qui existe déjà.

SEC-06	ÉLEVÉ	Dépendances vulnérables, dont <code>xlsx</code> exposé aux fichiers téléversés par les utilisateurs
CONSTAT	7 vulnérabilités <i>high</i> : <code>xlsx</code> 0.18.5 (pollution de prototype + ReDoS, aucun correctif publié sur npm), <code>pdfjs-dist</code> , <code>react-router</code> , <code>postcss</code> , <code>nanoid</code> , <code>undici</code> . <code>xlsx</code> parse dans le navigateur des fichiers fournis par l'utilisateur (<code>ImportPage</code>).	
PREUVE	<code>npm audit --json</code> · <code>app/src/pages/ImportPage.tsx</code> · <code>app/src/lib/excel-utils.ts</code>	
CORRECTIF	<code>npm audit fix</code> pour les six premières ; remplacer <code>xlsx</code> par <code>exceljs</code> ou par la distribution CDN maintenue de SheetJS (pinnée), ou déplacer le parsing dans une edge function avec limite de taille.	

SEC-07	ÉLEVÉ	Edge functions : CORS figé sur un domaine de test, contrôle de doublon d'email incorrect
CONSTAT	- Les origines autorisées se réduisent à <code>APP_URL</code> ou, par défaut, <code>https://projet-compta.zdouce-zz.workers.dev</code> : si <code>APP_URL</code> n'est pas posé sur chaque fonction, l'inscription et l'invitation échouent depuis <code>onusuite.com</code>. <code>auth-signup</code> appelle <code>auth.admin.listUsers()</code> sans pagination (50 par défaut) puis cherche l'email dans la page : au-delà de 50 comptes, le contrôle laisse passer les doublons et <code>createUser</code> renvoie une erreur non maîtrisée. - Le limiteur de débit est une <code>Map</code> en mémoire, dupliquée dans chaque fonction (alors que <code>_shared/rateLimit.ts</code> existe) : il se remet à zéro à chaque cold start et ne protège rien contre une attaque distribuée.	
PREUVE	<code>app/supabase/functions/auth-signup/index.ts:48-49, 125-135</code> · <code>app/supabase/functions/create-user/index.ts:60-64</code>	
CORRECTIF	Lire les origines depuis une variable <code>ALLOWED_ORIGINS</code> partagée ; utiliser <code>admin.getUserByEmail</code> (ou un <code>select</code> sur <code>auth.users</code> par la clé de service) ; factoriser le limiteur dans <code>_shared</code> et le sauvegarder dans une table ou Upstash si une vraie protection est voulue. Activer la protection anti-abus native de Supabase Auth (Turnstile/captcha) pour signup.	

Moyen : architecture, performance et hygiène

APP-02	MOYEN	Chaque rafraîchissement de jeton démonte toute l'application
--------	-------	---

CONSTAT	<code>onAuthStateChange</code> passe <code>loading</code> à <code>true</code> pour <i>tous</i> les événements, y compris <code>TOKEN_REFRESHED</code> (environ toutes les heures) et le retour sur l'onglet. <code>ProtectedRoute</code> affiche alors le spinner et démonte les pages : un formulaire de saisie en cours, un rapprochement bancaire à moitié fait, sont perdus. <code>loadUser</code> lit <code>user?.tenantId</code> dans une closure figée (<code>useCallback([], ...)</code>).
PREUVE	<code>app/src/lib/auth.tsx:221-227</code>
CORRECTIF	Ne recharger que sur <code>SIGNED_IN</code> , <code>SIGNED_OUT</code> et <code>USER_UPDATED</code> , sans toucher <code>loading</code> quand un utilisateur est déjà chargé.

SEC-08 **MOYEN** **Le contrôle d'accès par module est côté client et échoue « ouvert »**

CONSTAT	<code>useTenantModules</code> retourne <i>tous</i> les modules pendant le chargement et en cas d'erreur ; <code>ModuleGuard</code> et <code>useModuleAccess</code> décident dans le navigateur à partir de <code>module_roles</code> . Seules les tables de documents (migration 68) font respecter ces rôles en base. Le RBAC métier est donc contournable par appel direct à PostgREST pour tout ce qui n'est pas « documents ».
PREUVE	<code>app/src/lib/useTenantModules.ts:20, 77-80</code> · <code>app/src/components/ProtectedRoute.tsx:117-130</code>
CORRECTIF	Échouer fermé (aucun module tant que la liste n'est pas connue) ; étendre <code>has_module_access(module)</code> aux politiques RLS des tables de chaque module (une politique par table est suffisante : <code>tenant_id = current_tenant_id() AND has_module_access('hr')</code>).

PERF-01 **MOYEN** **Aucun chargement différé : la page d'accueil publique télécharge tout l'ERP**

CONSTAT	<code>App.tsx</code> importe statiquement 221 modules ; zéro <code>React.lazy</code> . Le chunk principal fait 3,39 Mo (non compressé), auxquels s'ajoutent 1,26 Mo de worker PDF, 460 Ko de pdf.js, 425 Ko de recharts et 422 Ko de xlsx. La landing page <code>/</code> est dans le même bundle.
PREUVE	<code>app/src/App.tsx:1-221</code> · <code>app/dist/assets/</code>
CORRECTIF	Un <code>lazy()</code> par module métier (accounting, hr, stock...) avec <code>Suspense</code> dans <code>ProtectedLayout</code> ; sortir la landing dans une entrée Vite séparée ou en HTML statique ; charger pdf.js et xlsx à la demande dans les pages d'import.

SEC-09 **MOYEN** **Sécurité de façade : anti-inspection, « anti-injection SQL » et sanitiseurs inutilisés**

CONSTAT	<code>antiInspection.ts</code> désactive le clic droit et F12 en production et affiche « Cette action est enregistrée », ce qui est faux. Cela n'arrête personne (le code est dans le bundle), casse les outils d'accessibilité et le support utilisateur, et la détection par taille de fenêtre se déclenche sur un écran partagé. <code>inputSanitizer.ts</code> : <code>sanitizeText</code> encode les entités HTML <i>avant stockage</i> (double échappement garanti au rendu React) et <code>detectSqlInjection</code> rejette des libellés légitimes (« Facture -- annulée »). Aucune des deux n'est appelée (0 référence). <code>sanitizeFilename</code> est dupliquée avec <code>fileSecurity.ts</code>. - Le limiteur de connexions dans <code>localStorage</code> / <code>sessionStorage</code> se contourne en un clic ; Supabase Auth a déjà un limiteur serveur.
PREUVE	<code>app/src/lib/antiInspection.ts</code> · <code>app/src/lib/inputSanitizer.ts:13-32, 142-158</code> · <code>app/src/lib/auth.tsx:232-252</code>

CORRECTIF

Supprimer les trois. Conserver `fileSecurity.ts` (utile) et valider les entrées avec un schéma (`zod`) au lieu de « nettoyer ».

CODE-01

MOYEN

Typage abandonné à la frontière des données

CONSTAT

1 262 `any` dans `src/`, dont 104 fonctions de requête qui renvoient `as any[]`. Le client Supabase n'est pas typé (`createClient` sans `Database`), donc aucune faute de colonne n'est détectée à la compilation. Le test « 0 `as any` » échoue sur 118 pages.

PREUVE

```
app/src/lib/supabase.ts:11 · grep -c "as any\[\\]"
src/lib/queries/*.ts
```

CORRECTIF

`supabase gen types typescript --linked > src/types/database.ts` en CI, `createClient<Database>`, puis retirer les casts module par module. Activer `noImplicitAny` est déjà le cas ; ajouter la règle `oxlint typescript/no-explicit-any` en avertissement puis en erreur.

CODE-02

MOYEN

Fichiers monolithiques et pages regroupées par « phase » plutôt que par domaine

CONSTAT

`types/index.ts` 4 205 lignes, `queries/accounting.ts` 3 910, `queries/misc.ts` 2 727, `Sidebar.tsx` 1 599 (données de navigation incluses), `App.tsx` 631. Les fichiers `Phase2Pages.tsx` à `Phase7DPages.tsx`, `sprintDE.ts`, `sprintH.ts` contiennent 4 à 15 pages chacun, nommés d'après le planning et non le métier. 127 `window.confirm / prompt`, 278 `console.*`.

CORRECTIF

Réorganiser en `src/modules/{accounting,hr,stock,...}/{pages,queries,types}` ; extraire `navModules` dans un fichier de données ; un composant `ConfirmDialog` et un logger unique branché sur Sentry.

TEST-01

MOYEN

2 212 tests qui ne testent pas la base

CONSTAT

Les tests de requêtes remplacent la chaîne Supabase par un mock : ils vérifient que le code appelle `.eq('tenant_id')`, jamais que RLS, les triggers ou les fonctions SQL existent. Un seul test de page, trois specs Playwright, aucun test de base de données. C'est pourquoi les RPC manquants (APP-01) et les politiques ouvertes (SEC-02) passent inaperçus.

CORRECTIF

Un job CI avec `supabase start` : pgTAP pour l'isolation par tenant et les triggers comptables ; quelques tests d'intégration Vitest contre la base locale pour les RPC critiques.

OPS-03

MOYEN

Hygiène du dépôt

CONSTAT

- Suivis par git : `coverage/` (1,5 Mo), `mirror-data/` (exports CSV/JSON/SQL de données métier, `users.csv` avec emails), `supabase/.temp/`, `mirror-daemon/config.json`, deux PNG de 3 Mo, un fichier nommé `new`, `tr_remaining_pages.py`.
- Vingt-trois fichiers `PLAN-*.md` et `SAGE-*.md` (≈1 Mo) à la racine ; `README.md` est encore le gabarit Vite.
- `pg` est dans les `dependencies` du front (utilisé seulement par le script de migration) ; `@sentry/react` est en `devDependencies` alors qu'il s'exécute en production.

• Deux `netlify.toml` identiques, un `wrangler.jsonc`, un `nginx.conf` : trois cibles de déploiement pour une seule utilisée.

CORRECTIF Nettoyer avec `git rm --cached`, déplacer la documentation dans `docs/`, écrire un vrai README (stack, démarrage, migrations, déploiement), corriger `package.json`.

SEC-10 MOYEN **Le démon miroir copie toute la base sur un poste de travail, en clair**

CONSTAT `mirror-daemon` se connecte avec un compte email/mot de passe stocké dans `config.json`, télécharge périodiquement les tables et les écrit dans `~/Desktop/Compta-Donnees` avec `encrypt: false` par défaut.

PREUVE `app/mirror-daemon/config.json · app/mirror-daemon/daemon.mjs:48-52, 108`

CORRECTIF Chiffrement obligatoire (refuser de démarrer sinon), compte de service dédié en lecture seule et rôle `auditor`, journalisation des exports dans `audit_log`. Documenter ce mécanisme dans la politique RGPD.

APP-03 MOYEN **Marquer une facture « payée » ne crée ni règlement ni écriture**

CONSTAT `handleMarkPaid` met `status: 'paid'`, `amount_paid` et `amount_due: 0` sans créer de `customer_payments` ni de mouvement bancaire. La balance âgée, la trésorerie et la comptabilité divergent immédiatement de la liste des factures.

PREUVE `app/src/pages/InvoicesPage.tsx:66-79`

CORRECTIF Retirer le bouton ou l'ouvrir sur le formulaire de règlement ; recalculer `amount_paid` par trigger à partir des règlements lettrés.

OPS-04 MOYEN **Le service worker met en cache les réponses de navigation quel que soit leur statut**

CONSTAT En mode `navigate`, toute réponse (y compris 404/500/redirection) est copiée dans le cache et servie hors ligne. Le cache-first sur tout GET same-origin masque aussi les mises à jour de `manifest.json` et des images non hachées.

PREUVE `app/public/sw.js:31-42, 45-55`

CORRECTIF Ne mettre en cache que `res.ok` ; limiter le cache-first à `/assets/` ; ou adopter `vite-plugin-pwa` qui génère un SW correct avec précache versionné.

Faible

ACC-03 FAIBLE **Calcul d'échéance sensible au fuseau horaire**

CONSTAT `calculateEcheance` construit une `Date` UTC depuis `'YYYY-MM-DD'`, manipule en heure locale puis repasse par `toISOString()` : selon le fuseau du poste, la date peut glisser d'un jour. La convention « fin de mois » (ajout des jours puis fin de mois) mérite d'être documentée car l'autre lecture existe.

PREUVE `app/src/lib/queries/accounting.ts:3264-3288`

CORRECTIF Utiliser `date-fns` (déjà en dépendance) : `addDays` , `endOfMonth` , `format(d, 'yyyy-MM-dd')` .

UX-01 **FAIBLE** Accessibilité peu travaillée

CONSTAT 44 attributs `aria-*` sur 395 fichiers ; tableaux triables, modales et combobox maison sans rôles ni gestion du focus ; les raccourcis clavier sont bloqués par l'anti-inspection.

CORRECTIF Audit axe-core sur les dix pages les plus utilisées ; composants `Modal` / `Combobox` conformes au pattern ARIA.

CODE-03 **FAIBLE** Exports en doublon exposés par `export *`

CONSTAT `createDocumentShare` et `getDocumentStats` sont définis dans `dematRh.ts` , `documents.ts` et `posDemat.ts` ; `queries/index.ts` ré-exporte tout par `export *` . Le nom résolu dépend de l'ordre des lignes et masque des fonctions aux sémantiques différentes.

CORRECTIF Préfixer par domaine (`createRhDocumentShare` , `createPosDocumentShare`) ou exporter par espace de noms.

OPS-05 **FAIBLE** Migrations : numérotation et runner fragiles

CONSTAT Numéros 38, 44 et 71 en double ; le runner exécute chaque fichier en un seul `query()` (impossible de localiser l'erreur), et enregistre les échecs sans arrêter la chaîne, donc les migrations suivantes s'appliquent sur un schéma partiel.

CORRECTIF Voir DB-01 : basculer sur `supabase migration` et arrêter à la première erreur.

UX-02 **FAIBLE** Fonctions exécutées sans retour utilisateur

CONSTAT Nombreux `catch(() => {})` et `console.error` sans toast (taux de change, chargement de factures, marquage « vu » d'un partage). L'utilisateur voit une liste vide et conclut qu'il n'y a pas de données.

CORRECTIF Un `useQuery` -like maison (ou TanStack Query) avec état d'erreur rendu par défaut dans `SkeletonTable` / `EmptyState` .

Ce qui est solide

- TypeScript strict qui compile sans erreur sur 100 000 lignes ; parité i18n fr/en/ar vérifiée en CI, support RTL.
- 192 tables ont de vraies politiques par tenant ; `audit_log` est immuable pour les clients ; 421 index sur `tenant_id` , 333 contraintes `CHECK` , 287 clés étrangères.
- Les edge fonctions vérifient le JWT, l'appartenance et le rôle admin dans le tenant cible avant d'utiliser la clé de service ; les politiques de stockage isolent par dossier tenant et par rôle de module (migration 68).
- Export FEC, extourne, Factur-X, moteur de taxes composables (`taxCalculator.ts`) : les briques réglementaires existent, il manque les garde-fous.
- Validation des fichiers téléversés par extension, MIME et octets magiques ; Sentry configuré avec masquage des replays.
- Le rapport de sécurité du 28 juillet avait déjà signalé le fallback de tenant et les politiques ouvertes de la migration 21 : le diagnostic existait, c'est la généralisation du correctif qui a manqué.

Plan de correction

Semaine 1 : arrêter l'hémorragie.

1. Rotation du mot de passe Postgres et des clés API ; purge de l'historique git ; gitLeaks en pre-commit (SEC-01).
2. Migration RLS générique + révocation des grants anon + requête de vérification en CI (SEC-02).
3. await setTenantId() , suppression du fallback « premier tenant », en-tête x-tenant-id en attendant le JWT (SEC-03).
4. Renommage des variables VITE_* sensibles, contrôle du bundle (SEC-04).
5. Déploiement conditionné au CI vert ; correction des 8 tests réels ; suppression du test « qualité » (OPS-01).
6. Une seule CSP, retrait de ipwho.is , taux de change via l'edge function (OPS-02).

Mois 1 : rendre les chiffres justes.

7. Baseline de schéma depuis la production, chaîne de migrations unique, tests pgTAP d'isolation (DB-01, TEST-01).
8. RPC post_journal_entry , triggers d'immuabilité et d'équilibre, contraintes d'unicité, transfert idempotent (ACC-01).
9. Agrégations SQL pour bilan, balance, grand livre, FEC ; pagination des listes (ACC-02).
10. Fonctions de stock manquantes ou trigger ; test CI « RPC appelés c RPC définis » (APP-01).
11. Paie : grilles du tenant, séquences, persistance des paramètres de calcul (PAY-01).
12. Pages publiques de paiement/partage via edge function avec token haché (SEC-05).
13. Mise à jour des dépendances, remplacement de xlsx (SEC-06) ; CORS et contrôle d'email des edge functions (SEC-07).

Trimestre : réduire la dette.

14. Tenant actif dans le JWT via Access Token Hook (SEC-03, forme définitive) et RBAC de module en RLS (SEC-08).
15. Code splitting par module, landing séparée (PERF-01) ; suppression de la sécurité de façade (SEC-09).
16. Types générés depuis la base, retrait progressif des any (CODE-01) ; réorganisation par domaine (CODE-02).
17. Nettoyage du dépôt, README réel, une seule cible de déploiement (OPS-03) ; démon miroir chiffré (SEC-10).

Améliorations recommandées au-delà des correctifs

- **Une couche d'accès aux données unique.** 1 125 fonctions écrites à la main répètent le même motif « getTenantId , from , eq , throw ». Un petit repo(table) typé qui applique tenant, pagination et gestion d'erreur réduirait queries/ de moitié et supprimerait une classe entière d'oublis.
- **TanStack Query** pour le cache, l'invalidation et les états d'erreur : la majorité des pages réimplémentent loading/error/refetch avec useState .
- **Périmètre.** Le produit couvre une surface de type Sage 100 + Odoo (plus de 300 routes). Avant d'ajouter des modules, geler la liste des fonctionnalités réellement livrables et retirer ou masquer celles qui reposent sur des RPC ou tables absents ; la promesse commerciale doit correspondre à ce que la base garantit.
- **Légalité comptable par pays.** Le pack de législation pilote la TVA mais ni la paie ni les règles de clôture ; formaliser par pack : format d'export légal (FEC ou équivalent), règles d'irréversibilité, plan de comptes, barèmes de paie datés.

- **Observabilité.** Sentry est câblé mais le DSN est vide ; y ajouter les erreurs des edge functions et un tableau de bord Supabase des requêtes lentes avant de monter en charge.
- **Runbook d'incident.** Rotation des secrets, restauration PITR, notification RGPD sous 72 h : à écrire maintenant, pendant qu'on rotate le mot de passe de SEC-01.

Méthode : lecture du code source, des 83 scripts SQL et des 6 edge functions ; exécution locale de `tsc` , `oxlint` , `vitest` , `npm audit` et `check-i18n.mjs` ; analyse du bundle `dist/` du 5 août. L'état réel des politiques en production n'a pas été interrogé (aucun accès base n'a été utilisé) : les constats SQL décrivent ce que le dépôt déclare, et DB-01 explique pourquoi cela peut différer de la production.